

Migracion Rick and Morty - Kotlin a Flutter

Este documento es la guía paso a paso de la migración de la app Android nativa Rick and Morty (construida con Kotlin, arquitectura MVVM, Retrofit para las llamadas HTTP, Room para la base de datos local y Firebase para autenticación y favoritos) hacia Flutter, manteniendo la misma arquitectura y las mismas funcionalidades. Cada paso muestra el código completo de cada archivo: cuando un archivo se modifica se ve el código anterior completo y el código nuevo completo, con una explicación de exactamente qué cambió y por qué.

Repositorio: <https://github.com/OscarLeoSanchez/app-rick-morty-flutter.git>

Paso 1 - Crear el proyecto Flutter

Comandos ejecutados

BASH

```
flutter create --org com.iue --platforms web,android,ios --empty rick_and_morty_flutter
cd rick_and_morty_flutter
```

Que hace cada opcion

`flutter create` genera la estructura base de un proyecto Flutter con todos los archivos necesarios para compilar.

`--org com.iue` define el identificador de organización que se usa para el package name de Android (`com.iue.rick_and_morty_flutter`) y el bundle ID de iOS (`com.iue.rickAndMortyFlutter`). Si no se especifica, Flutter usa `com.example` por defecto, lo que no es válido para publicar en stores.

`--platforms web,android,ios` genera las carpetas nativas para Android, iOS y Web. Sin esto Flutter también genera carpetas para Windows, Linux y macOS, que no se van a usar y agregan ruido al proyecto.

`--empty` crea el proyecto sin la app de ejemplo del contador, solo con la estructura mínima necesaria. Evita tener que eliminar manualmente el código de ejemplo en el siguiente paso.

`rick_and_morty_flutter` es el nombre del proyecto. Flutter requiere que el nombre esté en snake_case (solo minúsculas y guiones bajos).

Archivo generado por defecto - lib/main.dart

Con el flag `--empty` , Flutter genera este archivo con una estructura mínima sin el contador de ejemplo:

DART

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});
```

```

@override
Widget build(BuildContext context) {
  return const MaterialApp(
    home: Scaffold(
      body: Center(
        child: Text('Hello, World!'),
      ),
    ),
  );
}

```

Este código ya viene limpio. El siguiente paso lo adapta al proyecto.

Paso 2 - Limpiar main.dart

Por que se hace este cambio

Con `--empty` el archivo ya viene limpio y sin código de ejemplo innecesario. Los cambios que se hacen son: renombrar `MainApp` a `RickAndMortyApp`, agregar `debugShowCheckedModeBanner: false`, reemplazar el `home` temporal con el texto por la pantalla real, y preparar el `theme` para conectar el tema que se crea en el paso siguiente.

Codigo anterior (generado por flutter create)

DART

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Hello, World!'),
        ),
      ),
    );
  }
}

```

Codigo nuevo - lib/main.dart

DART

```

import 'package:flutter/material.dart';

void main() {
  runApp(const RickAndMortyApp());
}

```

```
class RickAndMortyApp extends StatelessWidget {
  const RickAndMortyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Rick and Morty',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(),
      home: const Scaffold(
        body: Center(
          child: Text('Rick and Morty App'),
        ),
      ),
    );
  }
}
```

Que cambio y por que

`MyApp` pasa a llamarse `RickAndMortyApp`. En Dart los nombres de clases deben describir su función; como esta clase representa la aplicación completa, el nombre debe reflejar el proyecto.

`MyHomePage` y `_MyHomePageState` se eliminan por completo. Esas clases eran el ejemplo del contador que no se va a usar.

`debugShowCheckedModeBanner: false` oculta el banner rojo de “DEBUG” que Flutter muestra por defecto en la esquina superior derecha cuando la app corre en modo debug. Es un cambio visual que hace que la app se vea más limpia durante el desarrollo.

`theme: ThemeData()` se deja vacío por ahora porque el tema completo se define en el siguiente paso en un archivo separado.

`home:` apunta a un `Scaffold` temporal con un `Text` en el centro. Es un placeholder que confirma que la app arranca correctamente antes de agregar pantallas reales.

Paso 3 - Crear el sistema de temas globales

Por que se hace este cambio

En la app Kotlin los colores y estilos estaban en `res/values/colors.xml` y `res/values/themes.xml`. En Flutter se centraliza todo en una clase `AppColors` con constantes y una clase `AppTheme` que configura el `ThemeData`. Esto permite cambiar cualquier color de la app desde un solo lugar, sin tener que buscar valores hardcodeados en cada pantalla.

Archivo nuevo - lib/app_theme.dart

```
DART

import 'package:flutter/material.dart';

class AppColors {
  AppColors._();

  static const Color primary = Color(0xFF97CE4C);
```

```

static const Color secondary = Color(0xFF11B0C8);
static const Color background = Color(0xFF060512);
static const Color surface = Color(0xFF1A1A2E);
static const Color textPrimary = Color(0xFFFFFFFF);
static const Color textSecondary = Color(0xFFB0B0B0);
static const Color error = Color(0xFFCF6679);
static const Color alive = Color(0xFF97CE4C);
static const Color dead = Color(0xFFCF6679);
static const Color unknown = Color(0xFFB0B0B0);
}

class AppTheme {
  AppTheme._();

  static ThemeData get darkTheme ⇒ ThemeData(
    brightness: Brightness.dark,
    scaffoldBackgroundColor: AppColors.background,
    colorScheme: const ColorScheme.dark(
      primary: AppColors.primary,
      secondary: AppColors.secondary,
      surface: AppColors.surface,
      error: AppColors.error,
      onPrimary: Colors.black,
      onSecondary: Colors.black,
      onSurface: AppColors.textPrimary,
    ),
    appBarTheme: const AppBarTheme(
      backgroundColor: AppColors.background,
      foregroundColor: AppColors.primary,
      elevation: 0,
      centerTitle: true,
    ),
    cardTheme: CardThemeData(
      color: AppColors.surface,
      elevation: 4,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    ),
    textTheme: const TextTheme(
      titleLarge: TextStyle(
        color: AppColors.textPrimary,
        fontSize: 20,
        fontWeight: FontWeight.bold,
      ),
      titleMedium: TextStyle(
        color: AppColors.textPrimary,
        fontSize: 16,
        fontWeight: FontWeight.w600,
      ),
      bodyMedium: TextStyle(color: AppColors.textSecondary, fontSize: 14),
      labelSmall: TextStyle(color: AppColors.textSecondary, fontSize: 12),
    ),
  );
}

```

Explicacion del codigo

`AppColors._()` es un constructor privado. El guion bajo en Dart indica que algo es privado. Al hacer el constructor privado se impide que alguien cree una instancia de `AppColors` con `AppColors()`. La clase es solo un contenedor de constantes, no tiene sentido instanciarla.

`static const Color` hace que los valores sean constantes en tiempo de compilación y accesibles sin crear una instancia (`AppColors.primary` en lugar de `AppColors().primary`). Esto es exactamente equivalente a los colores definidos en `colors.xml` en Android.

`Color(0xFF97CE4C)` recibe un entero en hexadecimal de 32 bits. El formato es `0xFF` seguido del color en RGB. El `FF` inicial es el canal alpha (opacidad completa). Es el mismo valor hexadecimal que se usa en Android pero con el prefijo `0xFF` en lugar de `#`.

`CardThemeData` es el tipo correcto dentro de `ThemeData` a partir de Flutter 3.29. En versiones anteriores se usaba `CardTheme`, que en las versiones nuevas es solo el nombre del widget, no el tipo de dato para el tema. Usar `CardTheme` dentro de `ThemeData` genera un error de tipos.

`AppTheme._()` también es un constructor privado por la misma razón que `AppColors`.

`static ThemeData get darkTheme` es un getter estático. Se accede con `AppTheme.darkTheme` sin paréntesis, como si fuera una propiedad. Retorna un `ThemeData` completamente configurado con todos los valores del diseño.

Desde cualquier widget en la app se puede acceder al tema con

`Theme.of(context).textTheme.titleLarge` o `Theme.of(context).colorScheme.primary`, lo que evita repetir estilos manualmente en cada pantalla.

Refactorización de lib/main.dart

Por que se refactoriza

Se conecta el tema recién creado y se apunta al `CharactersScreen` que se va a crear en el siguiente paso.

Código anterior

DART

```
import 'package:flutter/material.dart';

void main() {
  runApp(const RickAndMortyApp());
}

class RickAndMortyApp extends StatelessWidget {
  const RickAndMortyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Rick and Morty',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(),
      home: const Scaffold(
        body: Center(
          child: Text('Rick and Morty App'),
        ),
      ),
    );
  }
}
```

Código nuevo - lib/main.dart

DART

```
import 'package:flutter/material.dart';
import 'app_theme.dart';
import 'characters_screen.dart';

void main() {
  runApp(const RickAndMortyApp());
}

class RickAndMortyApp extends StatelessWidget {
  const RickAndMortyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Rick and Morty',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.darkTheme,
      home: const CharactersScreen(),
    );
  }
}
```

Que cambio

`theme: ThemeData()` pasa a `theme: AppTheme.darkTheme`. En lugar del tema vacío ahora se usa el tema completo que define todos los colores, tipografías y estilos de componentes. Con este cambio todos los widgets de la app heredan los estilos definidos en `app_theme.dart` automáticamente.

`home:` pasa de un `Scaffold` con un `Text` de placeholder a `CharactersScreen()`, que es la pantalla principal real de la app.

Se agregan los imports de `app_theme.dart` y `characters_screen.dart` para poder referenciar esas clases.

Paso 4 - Pantalla principal inicial

Por que se hace este cambio

Se crea el archivo de la pantalla de personajes con una estructura mínima para verificar que el tema aplica correctamente antes de agregar la lógica real.

Archivo nuevo - lib/characters_screen.dart

```
DART

import 'package:flutter/material.dart';

class CharactersScreen extends StatelessWidget {
  const CharactersScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Rick and Morty'),
      ),
      body: const Center(
```

```

        child: Text('Lista de personajes'),
      ),
    );
  }
}

```

Explicacion del codigo

`CharactersScreen` extiende `StatelessWidget` porque en esta etapa no tiene estado propio: solo muestra texto estático. Más adelante cuando se conecte la API real se convierte a `StatefulWidget`.

El `AppBar` usa automáticamente el `appBarTheme` definido en `AppTheme.darkTheme`. No se necesita especificar `backgroundColor` ni `foregroundColor` en cada pantalla porque el tema global los aplica. El texto `'Rick and Morty'` va a aparecer en el color `AppColors.primary` (verde) sobre el fondo oscuro.

El `Scaffold` usa el `scaffoldBackgroundColor` del tema sin necesidad de especificarlo explícitamente. El fondo oscuro se aplica solo.

Este archivo sirve para confirmar visualmente que la integración del tema funciona antes de agregar complejidad.

Paso 5 - Tarjeta de personaje

Por que se hace este cambio

En la app Kotlin cada item de la lista se manejaba con un `RecyclerView.ViewHolder` y un adapter. En Flutter se crea un widget reutilizable `CharacterCard` que recibe los datos del personaje y los muestra. Se construye primero la tarjeta sola para validar el diseño antes de armar la lista completa.

Archivo nuevo - lib/character_card.dart

DART

```

import 'package:flutter/material.dart';
import 'app_theme.dart';

class CharacterCard extends StatelessWidget {
  final String name;
  final String status;
  final String species;
  final String imageUrl;

  const CharacterCard({
    super.key,
    required this.name,
    required this.status,
    required this.species,
    required this.imageUrl,
  });

  Color _statusColor() {
    switch (status.toLowerCase()) {
      case 'alive':
        return AppColors.alive;
      case 'dead':
        return AppColors.dead;
      default:
        return AppColors.unknown;
    }
  }
}

```

```

    }
  }

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      child: SizedBox(
        height: 120,
        child: Row(
          children: [
            ClipRRect(
              borderRadius: const BorderRadius.only(
                topLeft: Radius.circular(12),
                bottomLeft: Radius.circular(12),
              ),
              child: Image.network(
                imageUrl,
                width: 120,
                height: 120,
                fit: BoxFit.cover,
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: Padding(
                padding: const EdgeInsets.symmetric(vertical: 12, horizontal: 8),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      name,
                      style: Theme.of(context).textTheme.titleMedium,
                    ),
                    const SizedBox(height: 6),
                    Row(
                      children: [
                        Container(
                          width: 10,
                          height: 10,
                          decoration: BoxDecoration(
                            color: _statusColor(),
                            shape: BoxShape.circle,
                          ),
                        ),
                        const SizedBox(width: 6),
                        Text(
                          '$status - $species',
                          style: Theme.of(context).textTheme.bodyMedium,
                        ),
                      ],
                    ),
                  ],
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```


Explicacion del codigo

`required` en los parámetros del constructor indica que son obligatorios. Si se intenta crear un `CharacterCard` sin pasar `name`, `status`, `species` o `imageUrl`, el compilador marca un error. Es equivalente a los parámetros sin valor por defecto en Kotlin.

`_statusColor()` es un método privado auxiliar (el guion bajo lo hace privado al archivo). Convierte el texto del estado en un color de la paleta de la app. `status.toLowerCase()` normaliza el texto para que `"Alive"`, `"alive"` y `"ALIVE"` sean equivalentes.

`SizedBox(height: 120)` le da una altura fija al `Row`. Sin esta restricción el `Row` no sabe cuánto espacio vertical tomar porque ninguno de sus hijos define una altura explícita, y Flutter lanza un error de layout. El `SizedBox` actúa como constraint.

`ClipRRect` recorta a su hijo usando bordes redondeados. `BorderRadius.only` con `topLeft` y `bottomLeft` aplica el borde solo en el lado izquierdo de la imagen, que es donde la imagen toca la tarjeta. El lado derecho no necesita borde porque la imagen no llega hasta ahí.

`Expanded` ocupa todo el espacio horizontal disponible que queda después de la imagen. Sin `Expanded` el texto intentaría ser tan ancho como su contenido y podría desbordarse o el `Row` no sabría cómo distribuir el espacio.

`Theme.of(context).textTheme.titleMedium` y `.bodyMedium` aplican los estilos tipográficos definidos en `AppTheme`. Al usar el tema en lugar de valores hardcoded, si se cambia el tamaño o color en `app_theme.dart` se actualiza en todas las tarjetas automáticamente.

Paso 6 - Lista con datos de prueba

Por que se hace este cambio

Antes de conectar la API real se prueba el layout de la lista con datos fijos para validar que la tarjeta funciona correctamente dentro de un `ListView.builder`.

Refactorizacion de lib/characters_screen.dart

Por que se refactoriza

Se reemplaza el texto de placeholder por una lista real con `ListView.builder` y datos de prueba.

Codigo anterior

DART

```
import 'package:flutter/material.dart';

class CharactersScreen extends StatelessWidget {
  const CharactersScreen({super.key});

  @override
  Widget build(BuildContext context) {
```

```

return Scaffold(
  appBar: AppBar(
    title: const Text('Rick and Morty'),
  ),
  body: const Center(
    child: Text('Lista de personajes'),
  ),
);
}
}

```

Codigo nuevo - lib/characters_screen.dart

DART

```

import 'package:flutter/material.dart';
import 'character_card.dart';

class CharactersScreen extends StatelessWidget {
  const CharactersScreen({super.key});

  final List<Map<String, String>> _mockCharacters = const [
    {
      'name': 'Rick Sanchez',
      'status': 'Alive',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/1.jpeg',
    },
    {
      'name': 'Morty Smith',
      'status': 'Alive',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/2.jpeg',
    },
    {
      'name': 'Summer Smith',
      'status': 'Alive',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/3.jpeg',
    },
    {
      'name': 'Abradolf Lincler',
      'status': 'unknown',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/7.jpeg',
    },
    {
      'name': 'Adjudicator Rick',
      'status': 'Dead',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/8.jpeg',
    },
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Rick and Morty'),
      ),
      body: ListView.builder(
        itemCount: _mockCharacters.length,
        itemBuilder: (context, index) {
          final character = _mockCharacters[index];
          return CharacterCard(
            name: character['name']!,
            status: character['status']!,

```

```

        species: character['species']!,
        imageUrl: character['image']!,
      ),
    ),
  );
}
}

```

Que cambio

`List<Map<String, String>>` es la lista de datos temporales. Cada personaje es un `Map` con claves de tipo `String` y valores de tipo `String`. Es el equivalente a una lista de mapas en Kotlin antes de tener la data class definida.

`ListView.builder` solo construye los items que son visibles en pantalla en un momento dado. Si la lista tiene 800 personajes pero solo se ven 6, solo se crean 6 widgets. Cuando el usuario hace scroll se crean los siguientes y se destruyen los anteriores. Este es exactamente el mismo comportamiento del `RecyclerView` de Android, que recicla los ViewHolder en lugar de crearlos todos al inicio.

`itemBuilder` es una función que se llama una vez por cada item visible. Recibe el `context` y el `index` del item, y debe retornar un widget.

`character['name']!` accede al valor del mapa con la clave `'name'`. El operador `!` al final es el operador de aserción de no-nulo de Dart. Le dice al compilador "confío en que este valor no es null". Si en tiempo de ejecución sí es null, la app lanza una excepción. En este caso es seguro usarlo porque los datos de prueba están definidos con todos los campos.

Paso 7 - Modelo de datos

Por que se hace este cambio

En la app Kotlin los modelos estaban en `domain/`. En Dart se crea una clase equivalente con un constructor `factory` para parsear el JSON de la API. Equivale a una `data class` con un `companion object` que tiene el método `fromJson`.

Archivo nuevo - lib/character.dart

DART

```

class Character {
  final int id;
  final String name;
  final String status;
  final String species;
  final String gender;
  final String image;
  final String origin;

  const Character({
    required this.id,
    required this.name,
    required this.status,
    required this.species,

```

```

    required this.gender,
    required this.image,
    required this.origin,
  });

  factory Character.fromJson(Map<String, dynamic> json) {
    return Character(
      id: json['id'],
      name: json['name'],
      status: json['status'],
      species: json['species'],
      gender: json['gender'],
      image: json['image'],
      origin: json['origin']['name'],
    );
  }

  factory Character.fromFirestore(Map<String, dynamic> json) {
    return Character(
      id: json['id'],
      name: json['name'],
      status: json['status'],
      species: json['species'],
      gender: json['gender'],
      image: json['image'],
      origin: json['origin'],
    );
  }
}

```

Explicación del código

const en el constructor indica que si todos los valores que se pasan son constantes en tiempo de compilación, el objeto también puede ser constante. Esto permite que Flutter optimice el árbol de widgets cuando los objetos no cambian.

factory es una palabra clave de Dart para constructores que no siempre retornan una nueva instancia o que necesitan hacer transformaciones antes de construir el objeto. Un constructor normal en Dart no puede tener código que retorne directamente un valor, pero un **factory** sí. Equivale a un método estático **fromJson** en el **companion object** de una **data class** en Kotlin.

json['origin']['name'] accede a un campo anidado. La API de Rick and Morty devuelve el origen del personaje como un objeto JSON: **"origin": {"name": "Earth", "url": " ... "}**. Por eso se necesita navegar dos niveles: primero el campo **origin** del JSON principal, y dentro de ese el campo **name**.

fromFirestore es un constructor **factory** separado porque en Firestore el origen se guardó como un **String** directo (no como un objeto anidado). Cuando se guarda el personaje como favorito, se serializa el **origin** como **character.origin** que ya es un **String**. Por eso al leer desde Firestore se usa **json['origin']** directamente sin **['name']**. Tener dos constructores separados hace explícito que el formato de los datos es diferente según la fuente.

Paso 8 - Servicio de API

Por que se hace este cambio

En la app Kotlin se usaba Retrofit con una interfaz anotada para las llamadas HTTP. En Flutter se usa el paquete `http` que hace lo mismo pero de forma más directa sin anotaciones. Primero se agrega la dependencia en `pubspec.yaml`.

Cambio en pubspec.yaml

Codigo anterior (seccion dependencies)

YAML

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
```

Codigo nuevo (seccion dependencies)

YAML

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
  cupertino_icons: ^1.0.8
```

Que cambio

Se agrega `http: ^1.2.2`. El símbolo `^` en el número de versión indica compatibilidad semántica: se acepta esa versión y cualquier versión superior que no rompa compatibilidad (en semver, cualquier versión `1.x.x` donde `x` sea mayor o igual al número especificado). Después de editar el archivo se ejecuta `flutter pub get` para descargar el paquete y actualizar `pubspec.lock`.

Archivo nuevo - lib/character_service.dart

DART

```
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'character.dart';

class CharacterService {
  static const String _baseUrl = 'https://rickandmortyapi.com/api/character';

  Future<List<Character>> getCharacters({int page = 1}) async {
    final url = Uri.parse('${_baseUrl}?page=$page');

    final response = await http.get(url);
    if (response.statusCode == 200) {
      final Map<String, dynamic> data = jsonDecode(response.body);
      final List results = data['results'];
      return results.map((json) => Character.fromJson(json)).toList();
    }
  }
}
```

```

    } else {
      throw Exception('Failed to load characters: ${response.statusCode}');
    }
  }
}

```

Explicación del código

`import 'package:http/http.dart' as http` importa el paquete con un alias. Sin el alias `as http`, cuando se llama a `http.get(url)` el compilador no sabría si `get` es una función del paquete o de otro lugar. Con el alias queda claro que `http.get` es la función del paquete `http`.

`Future<List<Character>>` es el tipo de retorno de la función. `Future` indica que la operación es asíncrona y va a completarse en algún momento en el futuro. Es el equivalente directo de `suspend fun` en Kotlin.

`async/await` es la sintaxis para código asíncrono en Dart. `async` marca que la función es asíncrona, y `await` pausa la ejecución hasta que el `Future` se complete. Es idéntico en concepto a `async/await` de Kotlin con coroutines.

`Uri.parse()` convierte el String de la URL en un objeto `Uri`. El paquete `http` requiere `Uri` como argumento y no acepta String directamente. Esto previene URLs mal formadas.

`{int page = 1}` es un parámetro nombrado con valor por defecto. La llave indica que es nombrado (se llama con `getCharacters(page: 2)`). El `= 1` indica que si no se pasa el parámetro, la primera página se carga por defecto.

`jsonDecode(response.body)` convierte el String JSON de la respuesta HTTP en un `Map<String, dynamic>`. Es el equivalente de `Gson.fromJson()` o `Moshi.adapter().fromJson()` en Kotlin.

`data['results']` extrae el array de personajes. La API devuelve un JSON con la estructura `{ "info": { ... }, "results": [ ... ] }`. El campo `info` tiene metadatos de paginación y `results` es la lista de personajes.

`throw Exception(...)` lanza una excepción si el servidor responde con un código de error. La pantalla que llama a este método puede capturar la excepción con `try/catch` y mostrar un mensaje de error al usuario.

Paso 9 - CharactersScreen con API real

Por que se hace este cambio

`CharactersScreen` pasa de `StatelessWidget` a `StatefulWidget` porque ahora necesita manejar tres estados que cambian en el tiempo: cargando, datos listos y error. Un `StatelessWidget` no puede cambiar su estado interno porque no tiene mecanismo para notificarle a Flutter que debe redibujar.

Refactorizacion de lib/characters_screen.dart

Por que se refactoriza

Se reemplaza la lista de datos de prueba por una llamada real a la API. Se convierte a `StatefulWidget` para manejar el estado de carga.

Codigo anterior

DART

```
import 'package:flutter/material.dart';
import 'character_card.dart';

class CharactersScreen extends StatelessWidget {
  const CharactersScreen({super.key});

  final List<Map<String, String>> _mockCharacters = const [
    {
      'name': 'Rick Sanchez',
      'status': 'Alive',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/1.jpeg',
    },
    {
      'name': 'Morty Smith',
      'status': 'Alive',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/2.jpeg',
    },
    {
      'name': 'Summer Smith',
      'status': 'Alive',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/3.jpeg',
    },
    {
      'name': 'Abradolf Lincler',
      'status': 'unknown',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/7.jpeg',
    },
    {
      'name': 'Adjudicator Rick',
      'status': 'Dead',
      'species': 'Human',
      'image': 'https://rickandmortyapi.com/api/character/avatar/8.jpeg',
    },
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Rick and Morty'),
      ),
      body: ListView.builder(
        itemCount: _mockCharacters.length,
        itemBuilder: (context, index) {
          final character = _mockCharacters[index];
          return CharacterCard(
            name: character['name']!,
            status: character['status']!,
            species: character['species']!,
            imageUrl: character['image']!,
          );
        },
      ),
    );
  }
}
```

```

    },
  ),
);
}
}

```

Codigo nuevo - lib/characters_screen.dart

DART

```

import 'package:flutter/material.dart';
import 'character_card.dart';
import 'character_service.dart';
import 'character.dart';
import 'character_detail_screen.dart';

class CharactersScreen extends StatefulWidget {
  const CharactersScreen({super.key});

  @override
  State<CharactersScreen> createState() => _CharactersScreenState();
}

class _CharactersScreenState extends State<CharactersScreen> {
  final CharacterService _characterService = CharacterService();

  List<Character> _characters = [];
  bool _isLoading = true;
  String? _errorMessage;

  @override
  void initState() {
    super.initState();
    _loadCharacters();
  }

  Future<void> _loadCharacters() async {
    try {
      final characters = await _characterService.getCharacters();
      setState(() {
        _characters = characters;
        _isLoading = false;
      });
    } catch (e) {
      setState(() {
        _errorMessage = e.toString();
        _isLoading = false;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return _buildBody();
  }

  Widget _buildBody() {
    if (_isLoading) {
      return const Center(child: CircularProgressIndicator());
    }
    if (_errorMessage != null) {
      return Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(_errorMessage!, style: const TextStyle(color: Colors.red)),

```



```

        const SizedBox(height: 16),
        ElevatedButton(
          onPressed: _loadCharacters,
          child: const Text('Reintentar'),
        ),
      ],
    ),
  );
}
return ListView.builder(
  itemCount: _characters.length,
  itemBuilder: (context, index) {
    final character = _characters[index];
    return GestureDetector(
      onTap: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (context) =>
              CharacterDetailScreen(character: character),
          ),
        );
      },
      child: CharacterCard(
        name: character.name,
        status: character.status,
        species: character.species,
        imageUrl: character.image,
      ),
    ),
  ),
);
}
}
}

```

Que cambio

Un `StatefulWidget` en Flutter se divide siempre en dos clases. La primera, `CharactersScreen`, extiende `StatefulWidget` y es inmutable: solo declara el constructor y el método `createState`. La segunda, `_CharactersScreenState`, extiende `State<CharactersScreen>` y es donde vive el estado mutable y la lógica. Esta separación existe porque Flutter puede descartar y recrear el widget del árbol, pero el State sobrevive ese proceso.

`initState()` es el método del ciclo de vida de Flutter que equivale a `onCreate` en Android. Se llama una sola vez cuando el widget se inserta en el árbol. Es el lugar correcto para iniciar la carga de datos porque se ejecuta antes del primer `build`.

`_characters`, `_isLoading` y `_errorMessage` son las tres variables de estado. `_isLoading` arranca en `true` para mostrar el indicador de carga inmediatamente. `_errorMessage` es `String?` (nullable) porque no hay error al inicio.

`setState(() { ... })` es la función que le dice a Flutter que el estado cambió y que debe llamar a `build` de nuevo para redibujar la pantalla. Todo cambio a variables de estado que deba reflejarse en la UI debe hacerse dentro de un `setState`. Sin `setState`, las variables cambian en memoria pero la pantalla no se actualiza.

`_buildBody()` es un método auxiliar privado que decide qué widget mostrar según el estado actual. Esto mantiene el método `build` limpio y evita anidar muchos condicionales.

`GestureDetector` con `onTap` detecta toques en el widget hijo. `Navigator.push` agrega una nueva pantalla al stack de navegación. `MaterialPageRoute` envuelve la pantalla de destino y gestiona la animación de transición. Esta es la forma estándar de navegar a una pantalla nueva en Flutter, equivalente a `findNavController().navigate( ... )` en Android.

El `Scaffold` se quitó del `build` de `CharactersScreen`. Esto es intencional: en el Paso 17 se va a crear `HomeScreen` que envuelve a `CharactersScreen` dentro de su propio `Scaffold`. Si `CharactersScreen` tuviera su propio `Scaffold`, habría dos `Scaffold` anidados, lo que genera comportamientos incorrectos en el layout.

Paso 10 - Pantalla de detalle (version inicial)

Por que se hace este cambio

En la app Kotlin el detalle era `DetailFragment`. En Flutter se crea `CharacterDetailScreen` como `StatelessWidget` inicialmente porque solo muestra datos que recibe como parámetro. Después se convierte a `StatefulWidget` cuando se agrega la funcionalidad de favoritos.

Archivo nuevo - lib/character_detail_screen.dart

DART

```
import 'package:flutter/material.dart';
import 'character.dart';
import 'app_theme.dart';

class CharacterDetailScreen extends StatelessWidget {
  final Character character;

  const CharacterDetailScreen({
    super.key,
    required this.character,
  });

  Color _statusColor() {
    switch (character.status.toLowerCase()) {
      case 'alive':
        return AppColors.alive;
      case 'dead':
        return AppColors.dead;
      default:
        return AppColors.unknown;
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(character.name),
      ),
      body: SingleChildScrollView(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Image.network(
              character.image,
              width: double.infinity,
```

```

        height: 300,
        fit: BoxFit.cover,
      ),
      Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              character.name,
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 12),
            _infoRow(context, 'Estado', character.status, _statusColor()),
            _infoRow(context, 'Especie', character.species, null),
            _infoRow(context, 'Genero', character.gender, null),
            _infoRow(context, 'Origen', character.origin, null),
          ],
        ),
      ),
    ],
  ),
);
}

Widget _infoRow(BuildContext context, String label, String value, Color? valueColor) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 6),
    child: Row(
      children: [
        Text(
          '$label: ',
          style: Theme.of(context).textTheme.bodyMedium,
        ),
        Text(
          value,
          style: Theme.of(context).textTheme.bodyMedium?.copyWith(
            color: valueColor ?? AppColors.textPrimary,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  );
}

```

Explicacion del codigo

SingleChildScrollView envuelve el contenido para que sea desplazable verticalmente. Si el contenido del **Column** no cabe en la pantalla (por ejemplo en un teléfono pequeño), el usuario puede hacer scroll para ver el resto. Sin este widget el contenido se recortaría.

double.infinity en **width** le dice a la imagen que ocupe todo el ancho disponible. No es un número infinito real sino la manera de decirle al layout “expandite al máximo posible dentro de tus constraints”. El **Column** le provee un ancho finito, así que la imagen ocupa ese ancho completo.

_infoRow es un método auxiliar privado que construye una fila de información con un label y un valor. Se usa cuatro veces (estado, especie, género, origen) y evita repetir el mismo bloque de código cuatro veces. Recibe **Color?** (nullable) para el color del valor, porque no todos los campos tienen un color especial.

`copyWith` modifica solo los campos especificados de un `TextStyle` existente y deja el resto igual. En este caso se parte del estilo `bodyMedium` del tema y se cambia el color y el peso de la fuente para el texto del valor. Es equivalente a `copy()` en una `data class` de Kotlin.

`valueColor ?? AppColors.textPrimary` usa el operador `??` de Dart, que retorna el valor de la derecha si el de la izquierda es null. Si `valueColor` es null (para especie, género y origen), se usa el color blanco por defecto.

Paso 11 - Configurar Firebase

Por que se hace este cambio

La app Kotlin usaba Firebase Auth para el login y Firestore para los favoritos. Se replica la misma integración en Flutter usando FlutterFire, que es el conjunto oficial de paquetes Firebase para Flutter.

Comandos ejecutados

BASH

```
npm install -g firebase-tools
firebase login
dart pub global activate flutterfire_cli
flutterfire configure
```

`npm install -g firebase-tools` instala la CLI de Firebase globalmente. Esta herramienta permite autenticarse con Google y gestionar proyectos Firebase desde la terminal.

`firebase login` abre el navegador para autenticarse con la cuenta de Google que tiene acceso al proyecto Firebase. La sesión queda guardada localmente.

`dart pub global activate flutterfire_cli` instala la CLI de FlutterFire, que es una herramienta específica para conectar proyectos Flutter con Firebase. Se instala con `dart pub` (el gestor de paquetes de Dart) en lugar de `npm` porque es una herramienta Dart.

`flutterfire configure` detecta el proyecto Flutter actual, lista los proyectos Firebase disponibles en la cuenta autenticada, genera automáticamente el archivo `lib/firebase_options.dart` con la configuración para cada plataforma (Android, iOS, web). Este archivo no se toca manualmente porque contiene claves y IDs específicos de cada proyecto Firebase.

Cambio en pubspec.yaml

Codigo anterior (seccion dependencies)

YAML

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
  cupertino_icons: ^1.0.8
```

Codigo nuevo (seccion dependencies)

YAML

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.2
  firebase_core: ^3.6.0
  firebase_auth: ^5.3.0
  cloud_firestore: ^5.4.0
  cupertino_icons: ^1.0.8
```

Que cambio

Se agregan tres paquetes. `firebase_core` es obligatorio como base para cualquier servicio Firebase: inicializa la conexión con el servidor de Firebase. Sin este paquete los otros dos no funcionan. `firebase_auth` provee las funciones de autenticación con email y contraseña. `cloud_firestore` provee acceso a la base de datos Firestore para guardar y leer favoritos. Después de editar el archivo se ejecuta `flutter pub get` para descargar los tres paquetes.

Paso 12 - Inicializar Firebase en main.dart

Por que se hace este cambio

Firebase necesita inicializarse antes de que la app arranque. Esto requiere que `main()` sea asíncrona y espere a que Firebase esté listo antes de llamar a `runApp()`.

Refactorizacion de lib/main.dart

Por que se refactoriza

Se agrega la inicialización de Firebase y se cambia el `home` a `LoginScreen`.

Codigo anterior

DART

```
import 'package:flutter/material.dart';
import 'app_theme.dart';
import 'characters_screen.dart';

void main() {
  runApp(const RickAndMortyApp());
}

class RickAndMortyApp extends StatelessWidget {
  const RickAndMortyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Rick and Morty',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.darkTheme,
```

```

        home: const CharactersScreen(),
      );
    }
  }
}

```

Codigo nuevo - lib/main.dart

DART

```

import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
import 'app_theme.dart';
import 'login_screen.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  runApp(const RickAndMortyApp());
}

class RickAndMortyApp extends StatelessWidget {
  const RickAndMortyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Rick and Morty',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.darkTheme,
      home: const LoginScreen(),
    );
  }
}

```

Que cambio

`async` en `main` hace que la función sea asíncrona. Dart requiere declarar `async` explícitamente para poder usar `await` dentro de la función.

`WidgetsFlutterBinding.ensureInitialized()` inicializa el binding entre el framework de Flutter y el motor de renderizado. Normalmente Flutter hace esto automáticamente cuando llama a `runApp`, pero si se necesita usar servicios de Flutter (como Firebase) antes de `runApp`, hay que inicializarlo manualmente. Sin esta línea, `Firebase.initializeApp()` puede lanzar un error porque el binding no está listo.

`await Firebase.initializeApp( ... )` espera a que Firebase establezca la conexión con los servidores de Google antes de continuar. Sin el `await`, `runApp` se ejecutaría antes de que Firebase esté listo y cualquier llamada a Auth o Firestore fallaría.

`DefaultFirebaseOptions.currentPlatform` es una constante generada por `flutterfire configure` en `firebase_options.dart`. Detecta en qué plataforma está corriendo la app (Android, iOS) y retorna la configuración correspondiente (App ID, API keys, etc.).

`home: const LoginScreen()` reemplaza a `CharactersScreen()` porque ahora el flujo empieza por el login. Después de autenticarse exitosamente, el login navega a `HomeScreen`.

Paso 13 - Pantalla de Login

Por que se hace este cambio

En la app Kotlin el login era `LoginFragment` con Firebase Auth. En Flutter se crea `LoginScreen` con `TextFormField`, validación de formulario con `GlobalKey<FormState>` y `FirebaseAuth`.

Archivo nuevo - lib/login_screen.dart

DART

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'app_theme.dart';
import 'home_screen.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();

  bool _isLoading = false;
  String? _error;

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    if (!_formKey.currentState!.validate()) return;

    setState(() {
      _isLoading = true;
      _error = null;
    });

    try {
      await FirebaseAuth.instance.signInWithEmailAndPassword(
        email: _emailController.text.trim(),
        password: _passwordController.text.trim(),
      );
      if (mounted) {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => const HomeScreen()),
        );
      }
    } on FirebaseAuthException catch (e) {
      setState(() {
        _error = _errorMessage(e.code);
        _isLoading = false;
      });
    }
  }
}
```

```

String _errorMessage(String code) {
  switch (code) {
    case 'user-not-found':
      return 'No existe una cuenta con ese correo.';
    case 'wrong-password':
      return 'Contraseña incorrecta.';
    case 'invalid-email':
      return 'El correo no es válido.';
    case 'too-many-requests':
      return 'Demasiados intentos. Intenta más tarde.';
    default:
      return 'Ocurrió un error. Intenta de nuevo.';
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: SafeArea(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Form(
          key: _formKey,
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                'Rick and Morty',
                textAlign: TextAlign.center,
                style: Theme.of(context).textTheme.titleLarge?.copyWith(
                  color: AppColors.primary,
                  fontSize: 32,
                ),
              ),
              const SizedBox(height: 48),
              TextFormField(
                controller: _emailController,
                keyboardType: TextInputType.emailAddress,
                decoration: const InputDecoration(
                  labelText: 'Correo',
                  border: OutlineInputBorder(),
                ),
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Ingresa tu correo';
                  }
                  return null;
                },
              ),
              const SizedBox(height: 16),
              TextFormField(
                controller: _passwordController,
                obscureText: true,
                decoration: const InputDecoration(
                  labelText: 'Contraseña',
                  border: OutlineInputBorder(),
                ),
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'Ingresa tu contraseña';
                  }
                  if (value.length < 6) {
                    return 'Mínimo 6 caracteres';
                  }
                  return null;
                },
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

```


Explicacion del codigo

Firebase (como `'user-not-found'` o `'wrong-password'`).

`Navigator.pushReplacement` navega a `HomeScreen` y elimina `LoginScreen` del stack de navegación al mismo tiempo. Si se usara `Navigator.push` en lugar de `pushReplacement` , el usuario podría volver al login presionando el botón de atrás del dispositivo, lo cual no tiene sentido después de autenticarse.

`_isLoading ? null : _login` en `onPressed` deshabilita el botón cuando `_isLoading` es true. En Flutter, pasar `null` a `onPressed` hace que el botón quede visualmente deshabilitado y no responda a toques. Esto evita que el usuario presione el botón varias veces mientras se procesa el login.

Paso 14 - Servicio de favoritos

Por que se hace este cambio

En la app Kotlin los favoritos se guardaban en Firestore con el UID del usuario como clave. Se replica la misma estructura en Flutter con `FavoritesService` .

Archivo nuevo - lib/favorites_service.dart

DART

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'character.dart';

class FavoritesService {
  final _firestore = FirebaseFirestore.instance;
  final _auth = FirebaseAuth.instance;

  String get _userId => _auth.currentUser!.uid;

  CollectionReference get _favoritesRef =>
    _firestore.collection('users').doc(_userId).collection('favorites');

  Future<void> addFavorite(Character character) async {
    await _favoritesRef.doc(character.id.toString()).set({
      'id': character.id,
      'name': character.name,
      'status': character.status,
      'species': character.species,
      'gender': character.gender,
      'image': character.image,
      'origin': character.origin,
    });
  }

  Future<void> removeFavorite(int characterId) async {
    await _favoritesRef.doc(characterId.toString()).delete();
  }

  Future<bool> isFavorite(int characterId) async {
    final doc = await _favoritesRef.doc(characterId.toString()).get();
    return doc.exists;
  }

  Stream<List<Character>> getFavorites() {
    return _favoritesRef.snapshots().map((snapshot) {
      return snapshot.docs.map((doc) {
        return Character.fromFirestore(doc.data() as Map<String, dynamic>);
      }).toList();
    });
  }
}
```

```
    });  
  }  
}
```

Explicacion del codigo

La ruta `users/{uid}/favorites` separa los favoritos de cada usuario. Si se guardaran todos en una colección plana, los favoritos de un usuario serían visibles para todos. Con esta estructura cada usuario tiene su propia subcolección de favoritos.

`String get _userId` es un getter que retorna el UID del usuario autenticado en el momento en que se llama. Se define como getter y no como campo porque `_auth.currentUser` puede cambiar durante la vida del servicio (si el usuario cierra sesión y vuelve a iniciar con otra cuenta).

`CollectionReference get _favoritesRef` es también un getter por la misma razón: usa `_userId` internamente, y como `_userId` es un getter, la referencia siempre apunta a la colección del usuario actual.

El `id` del personaje se convierte a String con `.toString()` porque las claves de documentos en Firestore son Strings. Usar el ID numérico como clave del documento garantiza que no puedan existir dos favoritos para el mismo personaje: si se intenta agregar el mismo personaje dos veces, el segundo `set` sobrescribe el primero.

`Stream<List<Character>>` en `getFavorites` hace que la pantalla de favoritos reciba actualizaciones en tiempo real. Cuando se agrega o elimina un favorito, Firestore emite un nuevo snapshot y la pantalla se actualiza automáticamente. Es el equivalente de observar un `LiveData` con `observe()` en Kotlin.

`fromFirestore` se usa en lugar de `fromJson` porque en Firestore el campo `origin` está guardado como un String directo. Cuando se guardó el personaje en `addFavorite`, se usó `character.origin` que ya es un String (el resultado del `json['origin']['name']` que se procesó en `Character.fromJson`). Si se usara `fromJson` al leer de Firestore, intentaría acceder a `json['origin']['name']` donde `origin` ya es un String, no un mapa, y lanzaría una excepción.

Paso 15 - Pantalla de favoritos

Por que se hace este cambio

En la app Kotlin los favoritos se mostraban en `FavoritesFragment` con un `RecyclerView` observando un `LiveData`. En Flutter se usa `StreamBuilder` que es el equivalente directo: escucha un `Stream` de Firestore y redibuja la pantalla automáticamente cuando hay cambios.

Archivo nuevo - lib/favorites_screen.dart

DART

```
import 'package:flutter/material.dart';  
import 'character.dart';  
import 'character_card.dart';  
import 'character_detail_screen.dart';  
import 'favorites_service.dart';  
  
class FavoritesScreen extends StatelessWidget {
```

```

const FavoritesScreen({super.key});

@override
Widget build(BuildContext context) {
  final service = FavoritesService();
  return StreamBuilder<List<Character>>(
    stream: service.getFavorites(),
    builder: (context, snapshot) {
      if (snapshot.connectionState == ConnectionState.waiting) {
        return const Center(child: CircularProgressIndicator());
      }
      if (snapshot.hasError) {
        return Center(child: Text(snapshot.error.toString()));
      }
      final favorites = snapshot.data ?? [];
      if (favorites.isEmpty) {
        return const Center(
          child: Text('No tenés favoritos todavía.'),
        );
      }
      return ListView.builder(
        itemCount: favorites.length,
        itemBuilder: (context, index) {
          final character = favorites[index];
          return GestureDetector(
            onTap: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) =>
                    CharacterDetailScreen(character: character),
                ),
            );
          },
            child: CharacterCard(
              name: character.name,
              status: character.status,
              species: character.species,
              imageUrl: character.image,
            ),
          );
        },
      );
    },
  );
}

```

Explicacion del codigo

`StreamBuilder<List<Character>>` es un widget que escucha un `Stream` y llama a su `builder` cada vez que el Stream emite un nuevo valor. `builder` recibe un `BuildContext` y un `AsyncSnapshot<List<Character>>` que contiene el estado actual del Stream.

`snapshot.connectionState` indica el estado de la conexión al Stream. `ConnectionState.waiting` significa que el Stream todavía no emitió ningún valor, es decir que los datos están cargando. En ese estado se muestra el indicador de progreso.

`snapshot.hasError` es `true` si el Stream emitió un error (por ejemplo un error de permisos en Firestore o un problema de conexión).

`snapshot.data ?? []` accede a los datos del último evento emitido por el Stream. Puede ser null si el Stream todavía no emitió datos (aunque ya pasó el estado `waiting`). El operador `??` garantiza una lista vacía en lugar de null para el caso en que `snapshot.data` sea null.

La pantalla de favoritos es `StatelessWidget` porque el estado real (la lista de personajes) vive en Firestore. El `StreamBuilder` gestiona la suscripción al Stream y el ciclo de vida automáticamente, por lo que no se necesita `StatefulWidget` ni `dispose` manual.

Paso 16 - Pantalla de detalle con favoritos

Por que se hace este cambio

Se agrega la funcionalidad de marcar y desmarcar favoritos en la pantalla de detalle. Para esto `CharacterDetailScreen` pasa de `StatelessWidget` a `StatefulWidget` porque ahora necesita recordar si el personaje es favorito y cambiar el ícono del corazón según ese estado.

Refactorizacion de lib/character_detail_screen.dart

Por que se refactoriza

Se convierte a `StatefulWidget` para manejar el estado del favorito y se agrega el botón en el AppBar.

Codigo anterior

DART

```
import 'package:flutter/material.dart';
import 'character.dart';
import 'app_theme.dart';

class CharacterDetailScreen extends StatelessWidget {
  final Character character;

  const CharacterDetailScreen({
    super.key,
    required this.character,
  });

  Color _statusColor() {
    switch (character.status.toLowerCase()) {
      case 'alive':
        return AppColors.alive;
      case 'dead':
        return AppColors.dead;
      default:
        return AppColors.unknown;
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(character.name),
      ),
      body: SingleChildScrollView(
        child: Column(
```

```

        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Image.network(
            character.image,
            width: double.infinity,
            height: 300,
            fit: BoxFit.cover,
          ),
          Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  character.name,
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                const SizedBox(height: 12),
                _infoRow(context, 'Estado', character.status, _statusColor()),
                _infoRow(context, 'Especie', character.species, null),
                _infoRow(context, 'Genero', character.gender, null),
                _infoRow(context, 'Origen', character.origin, null),
              ],
            ),
          ),
        ],
      ),
    ),
  );
}

Widget _infoRow(BuildContext context, String label, String value, Color? valueColor) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 6),
    child: Row(
      children: [
        Text(
          '$label: ',
          style: Theme.of(context).textTheme.bodyMedium,
        ),
        Text(
          value,
          style: Theme.of(context).textTheme.bodyMedium?.copyWith(
            color: valueColor ?? AppColors.textPrimary,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  );
}
}

```

Codigo nuevo - lib/character_detail_screen.dart

DART

```

import 'package:flutter/material.dart';
import 'character.dart';
import 'app_theme.dart';
import 'favorites_service.dart';

class CharacterDetailScreen extends StatefulWidget {
  final Character character;

  const CharacterDetailScreen({
    super.key,

```

```

        required this.character,
    });

    @override
    State<CharacterDetailScreen> createState() => _CharacterDetailScreenState();
}

class _CharacterDetailScreenState extends State<CharacterDetailScreen> {
    final FavoritesService _favoritesService = FavoritesService();
    bool _isFavorite = false;

    @override
    void initState() {
        super.initState();
        _checkFavorite();
    }

    Future<void> _checkFavorite() async {
        final result = await _favoritesService.isFavorite(widget.character.id);
        setState(() {
            _isFavorite = result;
        });
    }

    Future<void> _toggleFavorite() async {
        if (_isFavorite) {
            await _favoritesService.removeFavorite(widget.character.id);
        } else {
            await _favoritesService.addFavorite(widget.character);
        }
        setState(() {
            _isFavorite = !_isFavorite;
        });
    }

    Color _statusColor() {
        switch (widget.character.status.toLowerCase()) {
            case 'alive':
                return AppColors.alive;
            case 'dead':
                return AppColors.dead;
            default:
                return AppColors.unknown;
        }
    }

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(
                title: Text(widget.character.name),
                actions: [
                    IconButton(
                        icon: Icon(
                            _isFavorite ? Icons.favorite : Icons.favorite_border,
                            color: _isFavorite ? Colors.red : null,
                        ),
                        onPressed: _toggleFavorite,
                    ),
                ],
            ),
            body: SingleChildScrollView(
                child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                        Image.network(
                            widget.character.image,
                            width: double.infinity,
                            height: 300,

```

```

        fit: BoxFit.cover,
      ),
      Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              widget.character.name,
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 12),
            _infoRow(context, 'Estado', widget.character.status, _statusColor()),
            _infoRow(context, 'Especie', widget.character.species, null),
            _infoRow(context, 'Genero', widget.character.gender, null),
            _infoRow(context, 'Origen', widget.character.origin, null),
          ],
        ),
      ),
    ],
  ),
);
}

Widget _infoRow(BuildContext context, String label, String value, Color? valueColor) {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 6),
    child: Row(
      children: [
        Text(
          '$label: ',
          style: Theme.of(context).textTheme.bodyMedium,
        ),
        Text(
          value,
          style: Theme.of(context).textTheme.bodyMedium?.copyWith(
            color: valueColor ?? AppColors.textPrimary,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  );
}

```

Que cambio

La clase `CharacterDetailScreen` pasa de extender `StatelessWidget` a `StatefulWidget`. Esto requiere agregar el método `createState()` que retorna una instancia de `_CharacterDetailScreenState`.

El campo `final Character character` se mueve de la clase del widget a la clase del `StatefulWidget`. En el State se accede a ese campo a través de `widget.character` en lugar de `character` directamente. Esta es la diferencia clave entre un `StatelessWidget` y un `StatefulWidget`: en el primero los parámetros se acceden directamente, en el segundo se acceden a través de `widget.propiedad`.

`_isFavorite` es la nueva variable de estado booleana. Arranca en `false` y se actualiza cuando se consulta Firestore en `_checkFavorite()` y cuando el usuario toca el botón en `_toggleFavorite()`.

`_checkFavorite()` se llama en `initState` para consultar Firestore cuando se abre la pantalla y mostrar el ícono correcto desde el inicio. Sin esta consulta el corazón siempre arrancaría vacío aunque el personaje ya esté

en favoritos.

`_toggleFavorite()` verifica el estado actual y llama al método correspondiente del servicio. Después del `await` llama a `setState` con el valor negado. La UI se actualiza inmediatamente sin necesitar otra consulta a Firestore.

`actions` en el `AppBar` es una lista de widgets que aparecen en el lado derecho de la barra.

`Icons.favorite` es el corazón relleno y `Icons.favorite_border` es el corazón vacío. El color rojo solo se aplica cuando `_isFavorite` es true.

Paso 17 - Navegacion principal con HomeScreen

Por que se hace este cambio

En la app Kotlin la navegación entre pantallas principales (lista y favoritos) se manejaba con Navigation Component y un `BottomNavigationView`. En Flutter se crea `HomeScreen` que contiene el `BottomNavigationBar` y el botón de cerrar sesión, y gestiona qué pantalla mostrar según el tab activo.

Archivo nuevo - lib/home_screen.dart

DART

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'characters_screen.dart';
import 'favorites_screen.dart';
import 'login_screen.dart';
import 'app_theme.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  int _currentIndex = 0;

  final List<Widget> _screens = const [CharactersScreen(), FavoritesScreen()];

  Future<void> _logout() async {
    await FirebaseAuth.instance.signOut();
    if (mounted) {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (context) => const LoginScreen()),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Rick and Morty'),
        actions: [
          IconButton(
            icon: const Icon(Icons.logout),
```

```

        tooltip: 'Cerrar sesión',
        onPressed: _logout,
      ),
    ],
  ),
  body: _screens[_currentIndex],
  bottomNavigationBar: BottomNavigationBar(
    currentIndex: _currentIndex,
    backgroundColor: AppColors.surface,
    selectedItemColor: AppColors.primary,
    unselectedItemColor: AppColors.textSecondary,
    onTap: (index) {
      setState(() {
        _currentIndex = index;
      });
    },
    items: const [
      BottomNavigationBarItem(
        icon: Icon(Icons.people),
        label: 'Personajes',
      ),
      BottomNavigationBarItem(
        icon: Icon(Icons.favorite),
        label: 'Favoritos',
      ),
    ],
  ),
),
);
}
}

```

Explicacion del codigo

`_currentIndex` es la variable de estado que registra qué tab está activo. Arranca en `0` (el tab de personajes).

`_screens` es la lista de pantallas que se van a mostrar según el tab. El índice de la lista corresponde al índice del tab: `_screens[0]` es `CharactersScreen` y `_screens[1]` es `FavoritesScreen`.

`body: _screens[_currentIndex]` muestra la pantalla correspondiente al tab activo. Cuando el usuario toca otro tab, `setState` actualiza `_currentIndex` y Flutter redibuja el `body` con la pantalla nueva.

`BottomNavigationBar` es el equivalente directo del `BottomNavigationView` de Android. `currentIndex` le dice cuál item está seleccionado, `onTap` recibe el índice del item tocado y `items` define los tabs con su ícono y etiqueta.

`FirebaseAuth.instance.signOut()` cierra la sesión en Firebase. Después del `await`, `Navigator.pushReplacement` navega a `LoginScreen` y elimina `HomeScreen` del stack. Usar `pushReplacement` es importante porque si el usuario volviera atrás después de cerrar sesión, llegaría a `HomeScreen` sin estar autenticado, lo que causaría errores en Firestore.

Como el `AppBar` y el `Scaffold` ahora viven en `HomeScreen`, se eliminó el `Scaffold` de `CharactersScreen` y `FavoritesScreen`. El método `build` de `CharactersScreen` retorna directamente `_buildBody()` sin `Scaffold`. Si una pantalla hija tuviera su propio `Scaffold`, habría dos `Scaffold` anidados: el de `HomeScreen` y el de la pantalla hija. Esto genera comportamientos inesperados en el layout porque el `Scaffold` interno anula configuraciones del externo.

Clonar y configurar el proyecto

El repositorio del proyecto está en:

<https://github.com/OscarLeoSanchez/app-rick-morty-flutter.git>

Cada estudiante debe crear su propio proyecto Firebase y conectarlo al código clonado.

Pasos

1. Clonar el repositorio

BASH

```
git clone https://github.com/OscarLeoSanchez/app-rick-morty-flutter.git
cd app-rick-morty-flutter
```

2. Instalar dependencias

BASH

```
flutter pub get
```

3. Crear el proyecto en Firebase Console

Ir a console.firebase.google.com con la cuenta de Google personal y crear un proyecto nuevo. Activar dos servicios:

- Authentication: ir a Authentication, luego Sign-in method y activar Email/Password
- Firestore: ir a Firestore Database, crear base de datos y seleccionar modo de prueba

Sin estos dos pasos el login y los favoritos no funcionan aunque el código esté correcto.

4. Conectar con Firebase

BASH

```
firebase login
flutterfire configure
```

`flutterfire configure` pregunta qué proyecto Firebase usar. Seleccionar el proyecto recién creado. El comando sobrescribe `lib/firebase_options.dart` con la configuración propia del proyecto de cada estudiante.

5. Correr la app

BASH

```
flutter run
```

Lo que no hay que hacer al clonar

No correr `flutter create` porque el proyecto ya existe. Correr `flutter create` dentro de la carpeta clonada sobrescribiría los archivos del proyecto.

No modificar las carpetas `android/` ni `ios/` manualmente. Esas carpetas son generadas por Flutter y contienen configuración de compilación nativa que no se toca a mano.

No compartir el archivo `firebase_options.dart` entre estudiantes. Cada uno genera el suyo con `flutterfire configure` porque contiene las credenciales del proyecto Firebase propio. Compartir ese archivo haría que todos los estudiantes escriban en la misma base de datos Firestore.